Practical 7

Aim: Performance Optimization Using Caching and Partitioning

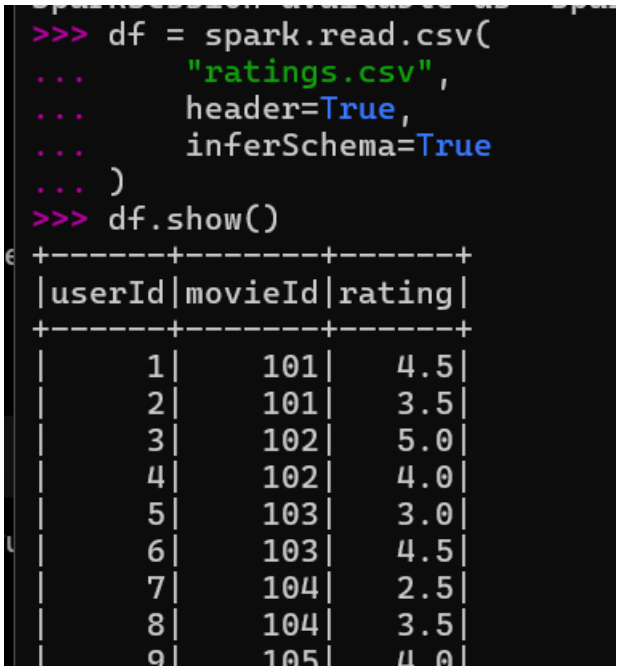
Problem definition: You are a Big Data Engineer at MovieFlix Technologies, a movie recommendation company serving millions of users worldwide. The recommendation engine repeatedly processes movie ratings data to generate personalized suggestions. Due to the increasing volume of user interactions, query execution time has become a major concern. Management wants to optimize the Spark application using Caching and Partitioning techniques to reduce execution time and improve scalability.

Observation / Output:

Task-1: Create Spark Session.



Task-2: Load ratings dataset into Spark DataFrame.



Task-3: Display dataset schema.

```
>>> df.printSchema()
root
 |-- userId: integer (nullable = true)
 |-- movieId: integer (nullable = true)
 |-- rating: double (nullable = true)

>>>
```

Task-4: Check the current number of partitions.

```
>>> df.rdd.getNumPartitions()
1
>>>
```

Task-5: Calculate average movie ratings.

```
>>> from pyspark.sql.functions import avg
>>> df.select(avg("rating")).show()
+-----------+
|avg(rating)|
+-----------+
|        4.0|
+-----------+

>>>
```

Task-6: Measure execution time before optimization.

```
>>> import time
>>> start = time.time()
...
... df.select(avg("rating")).show()
...
... end = time.time()
...
... before_cache = end - start
...
... print("Execution time before caching:", before_cache, "seconds")
...
+-----------+
|avg(rating)|
+-----------+
|        4.0|
+-----------+

Execution time before caching: 0.21927714347839355 seconds
```

Task-7: Apply cache() on the DataFrame.

```
>>> df.cache()
DataFrame[userId: int, movieId: int, rating: double]
```

Task-8: Trigger caching using count().

```
>>> df.count()
40
```

Task-9: Recalculate average ratings. Measure execution time after caching.

```
>>> start = time.time()
...
... df.select(avg("rating")).show()
...
... end = time.time()
...
... after_cache = end - start
...
... print("Execution time after caching:", after_cache, "seconds")
...
+-----------+
|avg(rating)|
+-----------+
|        4.0|
+-----------+

Execution time after caching: 0.16863656044006348 seconds
>>>
```

Task-10: Repartition the dataset. Display updated partition count.

```
>>> df4 = df.repartition(4)
>>> df4.rdd.getNumPartitions()
4
>>>
```

```
>>> start = time.time()
...
... df4.select(avg("rating")).show()
...
... end = time.time()
...
... print("Execution time with 4 partitions:",
...       end - start,
...       "seconds")
...
+-----------+
|avg(rating)|
+-----------+
|        4.0|
+-----------+

Execution time with 4 partitions: 0.25110650062561035 seconds
```

Task-11: Compare performance before and after optimization.

```
>>> print("Before caching:", before_cache, "seconds")
... print("After caching:", after_cache, "seconds")
...
Before caching: 0.15110230445861816 seconds
After caching: 0.16863656044006348 seconds
>>>
```

Task-12: Generate performance report.

```
>>> print("----- PERFORMANCE REPORT -----")
... print("Original partitions:", df.rdd.getNumPartitions())
... print("Partitions after repartition:", df4.rdd.getNumPartitions())
... print("Before caching:", before_cache, "seconds")
... print("After caching:", after_cache, "seconds")
...
----- PERFORMANCE REPORT -----
Original partitions: 1
Partitions after repartition: 4
Before caching: 0.15110230445861816 seconds
After caching: 0.16863656044006348 seconds
>>>
```

Key Questions /Analysis:

Q1. What is caching in Apache Spark and why does it improve performance?

Caching is a technique in Spark that stores a DataFrame or RDD in memory after it is computed. When the same data is used again, Spark can reuse the cached data instead of processing it from the beginning. This reduces repeated computation and can improve execution time.

Q2. What is the difference between cache() and persist()?

cache() stores the data using Spark's default storage level. persist() allows us to specify a particular storage level, such as memory or disk.

For example:

df.cache()

and:

df.persist(StorageLevel.DISK_ONLY)

So, cache() is simple and uses the default storage option, while persist() provides more control over storage.

Q3. How does Spark partition data internally?

Spark divides a large dataset into smaller units called **partitions**. These partitions can be distributed across the available processing resources. Each partition can be processed independently, allowing Spark to perform operations in parallel.

Q4. Why does repartition() improve scalability?

repartition() changes the number of partitions in a DataFrame. By distributing data across more partitions, Spark can use more parallel processing resources. This can help handle larger datasets efficiently and improve scalability.

Q5. How does lazy evaluation affect Spark performance?

Spark uses **lazy evaluation**, which means transformations are not executed immediately. Spark waits until an action such as count() or show() is called. This allows Spark to optimize the execution plan and avoid unnecessary operations, which can improve overall performance.

Conclusion:

In this practical, Spark was used to study performance optimization using caching and partitioning. The ratings dataset was loaded into a Spark DataFrame, its schema and partitions were examined, and average ratings were calculated. Execution time was measured before and after applying caching, and the dataset was repartitioned to observe the effect of different partition counts. The practical helped demonstrate how caching reduces repeated computation and how partitioning supports parallel processing and scalability in Spark.